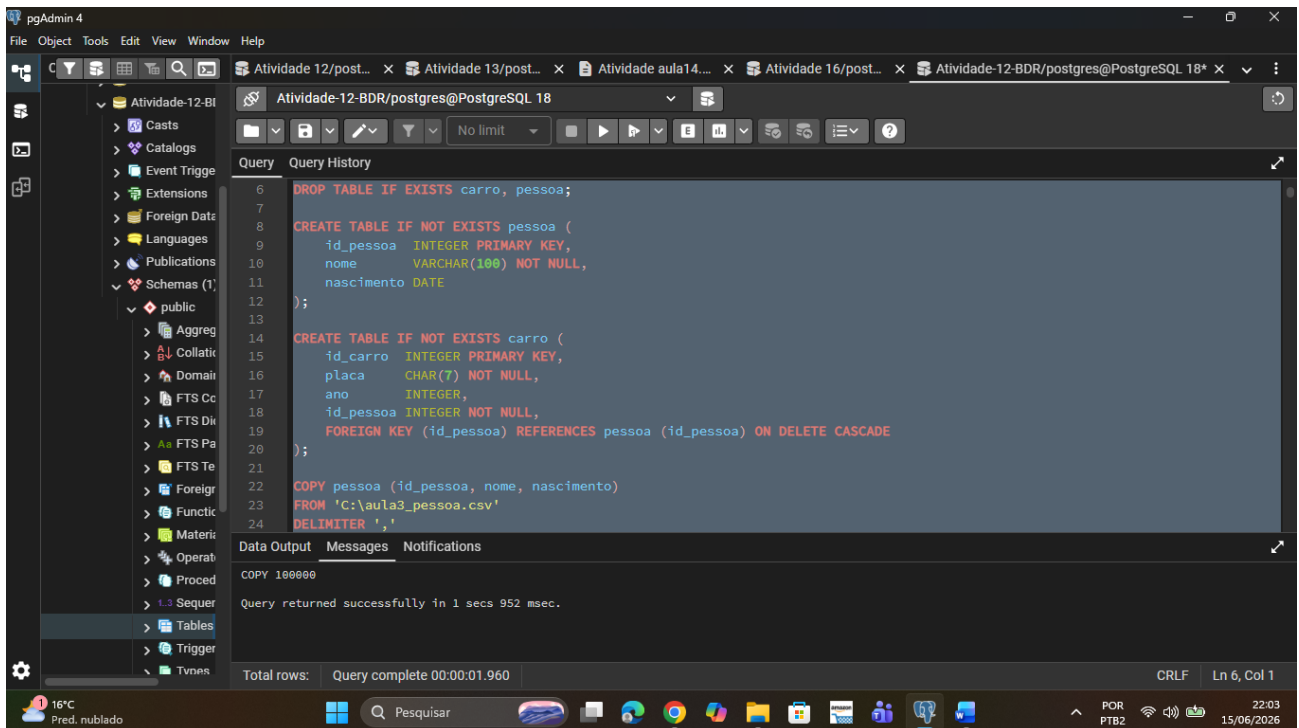


--Atividade-Aula12-BDR--

--Ronaldo de Avila Ribeiro Junior ----



-- Exercício 1 - Observando o impacto de um índice (B-tree) com EXPLAIN ANALYZE

-- Parte A - Consulta sem índice explícito

-- 1. Execute a consulta abaixo e registre o tipo de varredura e o tempo retornado:

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nome = 'Ana Silva';

pgAdmin 4

File Object Tools Edit View Window Help

Atividade 12/post... x Atividade 13/post... x Atividade aula14... x Atividade 16/post... x Atividade-12-BDR/postgres@PostgreSQL 18* x

Atividade-12-BDR/postgres@PostgreSQL 18

Query Query History

```
-- Parte A - Consulta sem indice explicito
-- 1. Execute a consulta abaixo e registre o tipo de varredura e o tempo retornado:
EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nome = 'Ana Silva';
```

Data Output Messages Notifications

Showing rows: 1 to 6 Page No: 1 of 1

QUERY PLAN	
text	
1	Seq Scan on pessoa (cost=0.00..1938.00 rows=6 width=23) (actual time=3.642..8.061 rows=5.00 loop=1)
2	Filter: ((nome)::text = 'Ana Silva')::text)
3	Rows Removed by Filter: 99995
4	Buffers: shared hit=688
5	Planning Time: 0.087 ms
6	Execution Time: 8.079 ms

Total rows: 6 Query complete 00:00:00.101 CRLF Ln 44, Col 1

16°C Pred. nublado 22:06 15/06/2026

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nome = 'Joao Santos';

pgAdmin 4

File Object Tools Edit View Window Help

Atividade 12/post... x Atividade 13/post... x Atividade aula14... x Atividade 16/post... x Atividade-12-BDR/postgres@PostgreSQL 18* x

Atividade-12-BDR/postgres@PostgreSQL 18

Query Query History

```
WHERE nome = 'Ana Silva';
-- 2. Execute uma segunda vez com outro nome comum, por exemplo:
EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nome = 'Joao Santos';
```

Data Output Messages Notifications

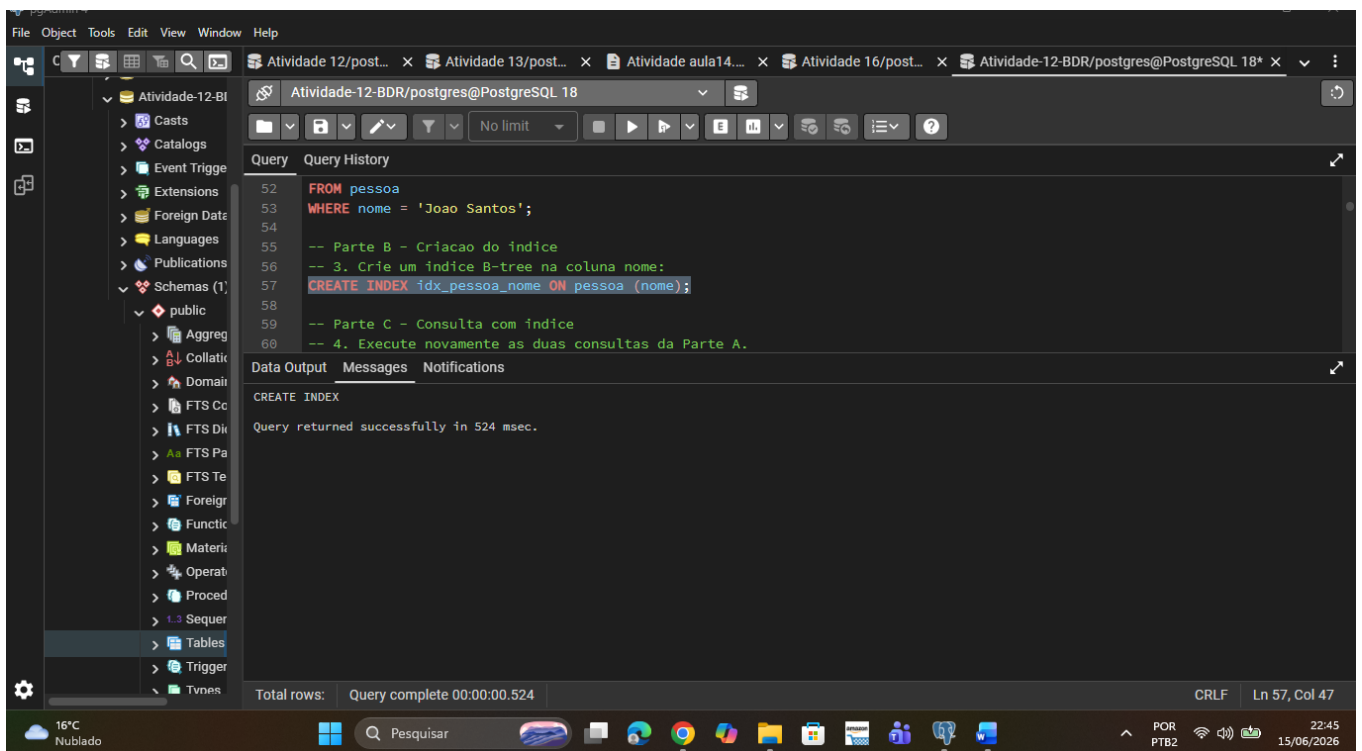
Showing rows: 1 to 6 Page No: 1 of 1

QUERY PLAN	
text	
1	Seq Scan on pessoa (cost=0.00..1938.00 rows=6 width=23) (actual time=8.894..8.894 rows=0.00 loop=1)
2	Filter: ((nome)::text = 'Joao Santos')::text)
3	Rows Removed by Filter: 100000
4	Buffers: shared hit=688
5	Planning Time: 0.159 ms
6	Execution Time: 8.919 ms

Total rows: 6 Query complete 00:00:00.092 CRLF Ln 53, Col 28

16°C Nublado 22:43 15/06/2026

-- Parte B - Criacao do índice



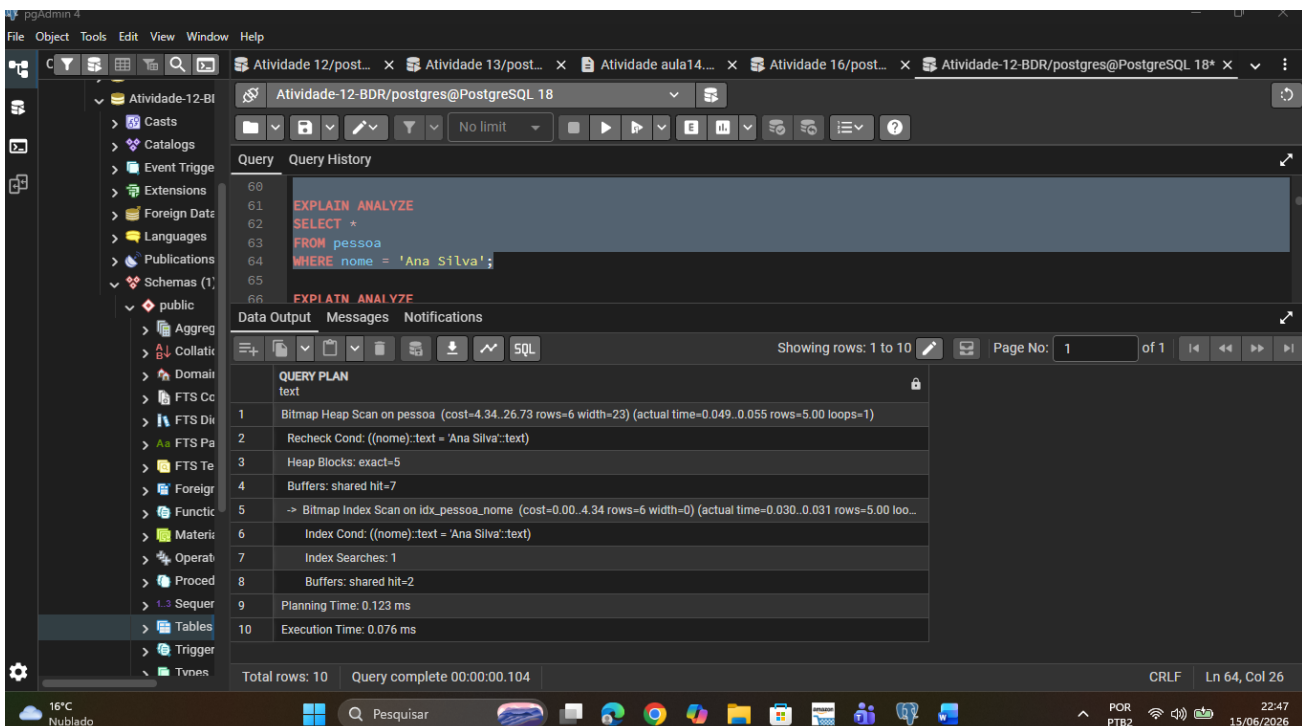
-- Parte C - Consulta com índice

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nome = 'Ana Silva';



EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nome = 'Joao Santos';

pgAdmin 4

File Object Tools Edit View Window Help

Atividade 12/post... x Atividade 13/post... x Atividade aula14... x Atividade 16/post... x Atividade-12-BDR/postgres@PostgreSQL 18*

Query Query History

```
65
66 EXPLAIN ANALYZE
67 SELECT *
68 FROM pessoa
69 WHERE nome = 'Joao Santos';
70
71 -- =====
```

Data Output Messages Notifications

Showing rows: 1 to 9 Page No: 1 of 1

QUERY PLAN
text
1 Bitmap Heap Scan on pessoa (cost=4.34..26.73 rows=6 width=23) (actual time=0.083..0.084 rows=0.00 loops=1)
2 Recheck Cond: ((nome)::text = 'Joao Santos':text)
3 Buffers: shared hit=1 read=1
4 -> Bitmap Index Scan on idx_pessoa_nome (cost=0.00..4.34 rows=6 width=0) (actual time=0.074..0.074 rows=0.00 loops=1)
5 Index Cond: ((nome)::text = 'Joao Santos':text)
6 Index Searches: 1
7 Buffers: shared hit=1 read=1
8 Planning Time: 0.115 ms
9 Execution Time: 0.101 ms

Total rows: 9 Query complete 00:00:00.102

Successfully run. Total query runtime: 102 ms

CRLF Ln 70, Col 1

16°C Nublado

Pesquisar

POR PTB2

22:48 15/06/2026

-- Parte D - Responda, de forma objetiva:

- Qual plano foi utilizado antes do indice?

R: Seq Scan

- Qual plano foi utilizado após o indice?

R: Bitmap Heap Scan

- Houve reducao no tempo de execucao?

R: Sim, houve uma redução de 7ms para 0,07ms

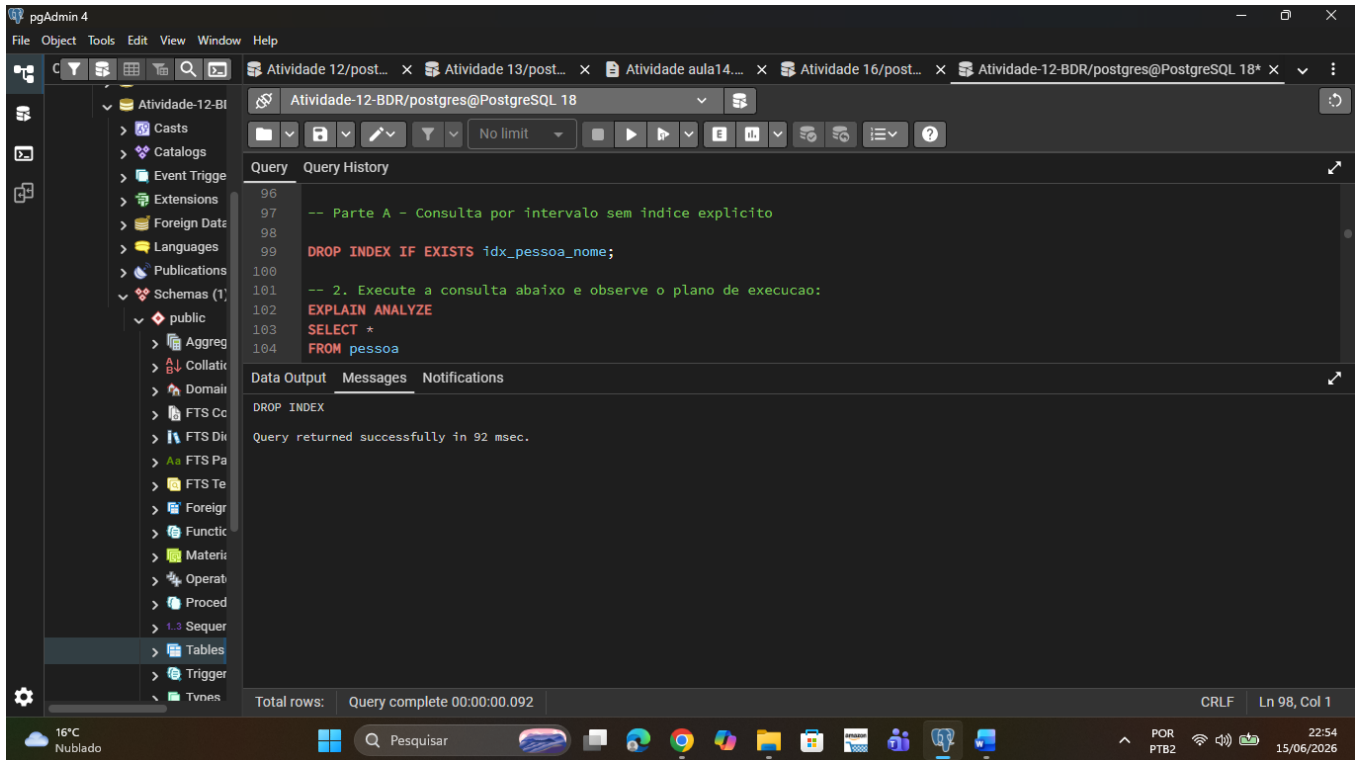
- O indice foi utilizado em ambos os nomes testados? Justifique com base na saída do plano.

R: Sim, pois a condicao nome = 'valor' e altamente seletiva em ambos os casos, fazendo o otimizador preferir o indice

-- Exercício 2 - Índice, Seletividade e Decisão do Otimizador

-- Parte A - Consulta por intervalo sem índice explícito

DROP INDEX IF EXISTS idx_pessoa_nome;

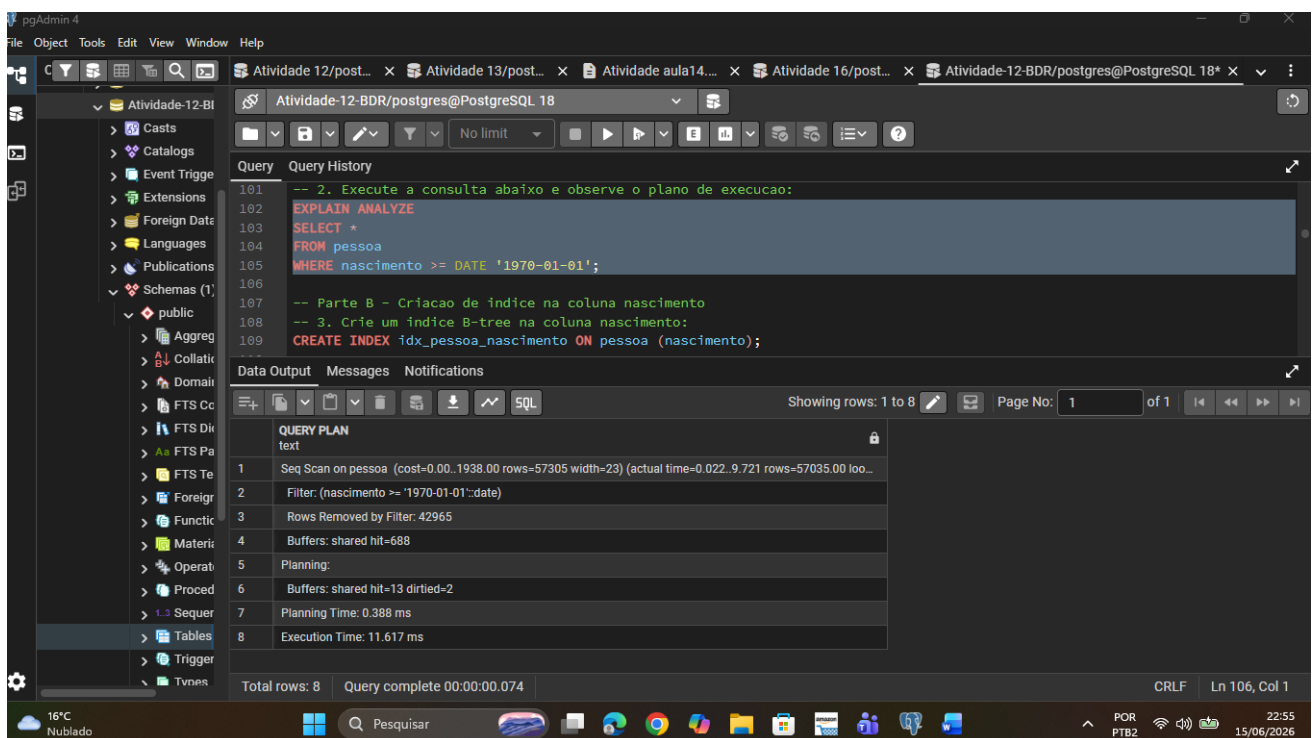


EXPLAIN ANALYZE

SELECT *

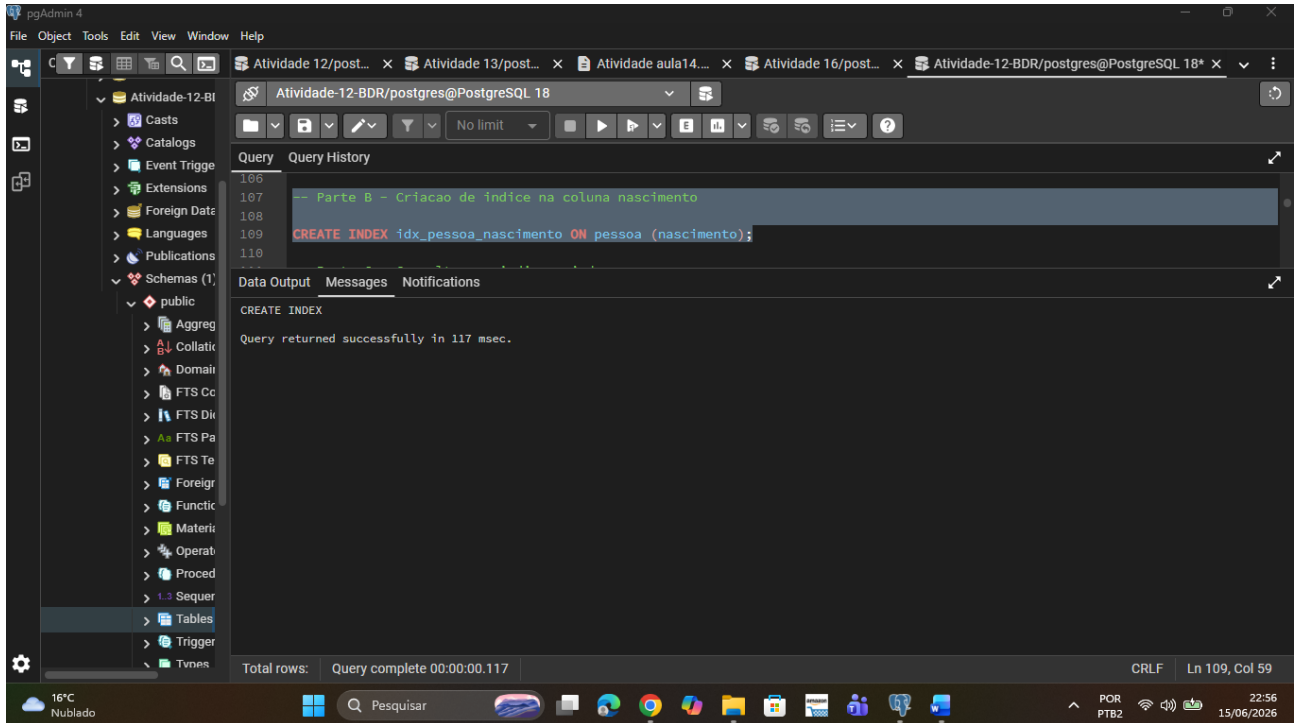
FROM pessoa

WHERE nascimento >= DATE '1970-01-01';



-- Parte B - Criacao de indice na coluna nascimento

CREATE INDEX idx_pessoa_nascimento ON pessoa (nascimento);



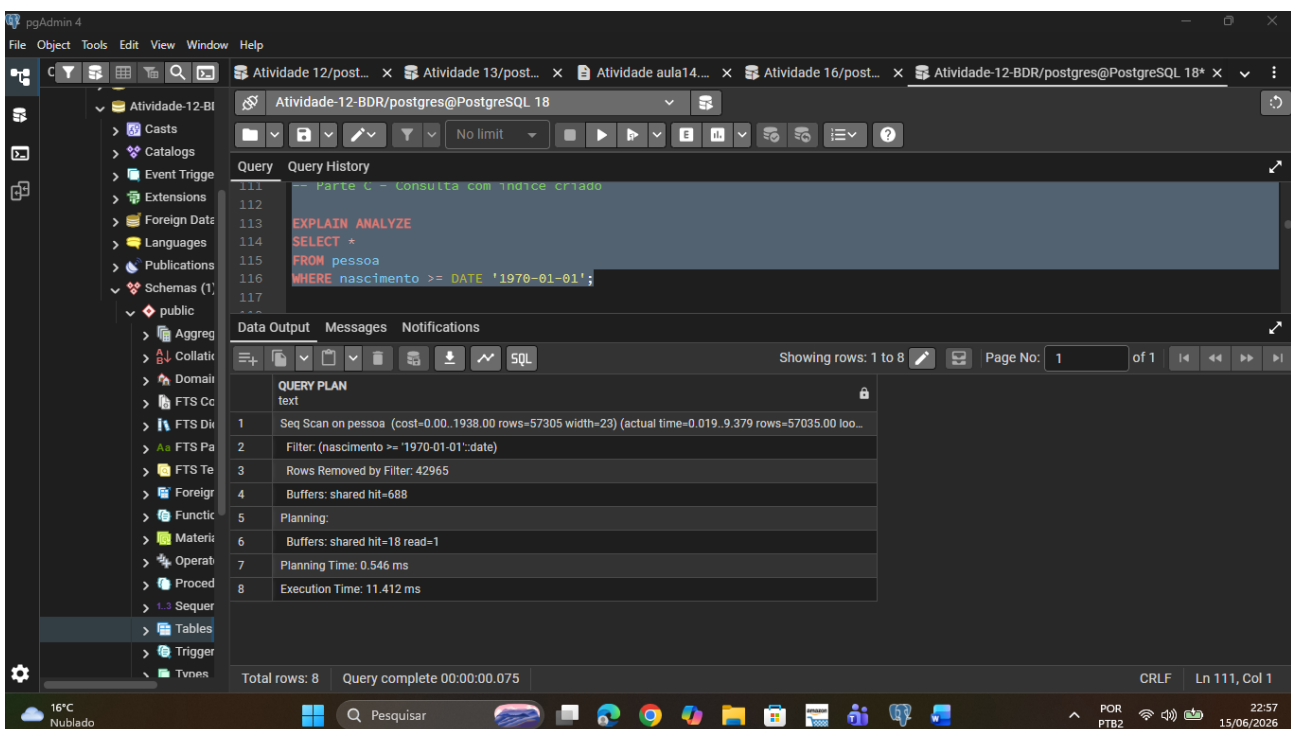
-- Parte C - Consulta com indice criado--

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nascimento >= DATE '1970-01-01';



- Parte D - Responda, de forma objetiva

- O índice criado foi utilizado pelo PostgreSQL?

R: Não, o SGBD não utilizou.

- O plano de execução mudou após a criação do índice?

R: Não, continuou sendo Seq Scan porque a condição `>= DATE '1970-01-01'` retorna muitas linhas (baixa seletividade)

- Houve melhora significativa no tempo de execução?

R: Não, houve um aumento no tempo.

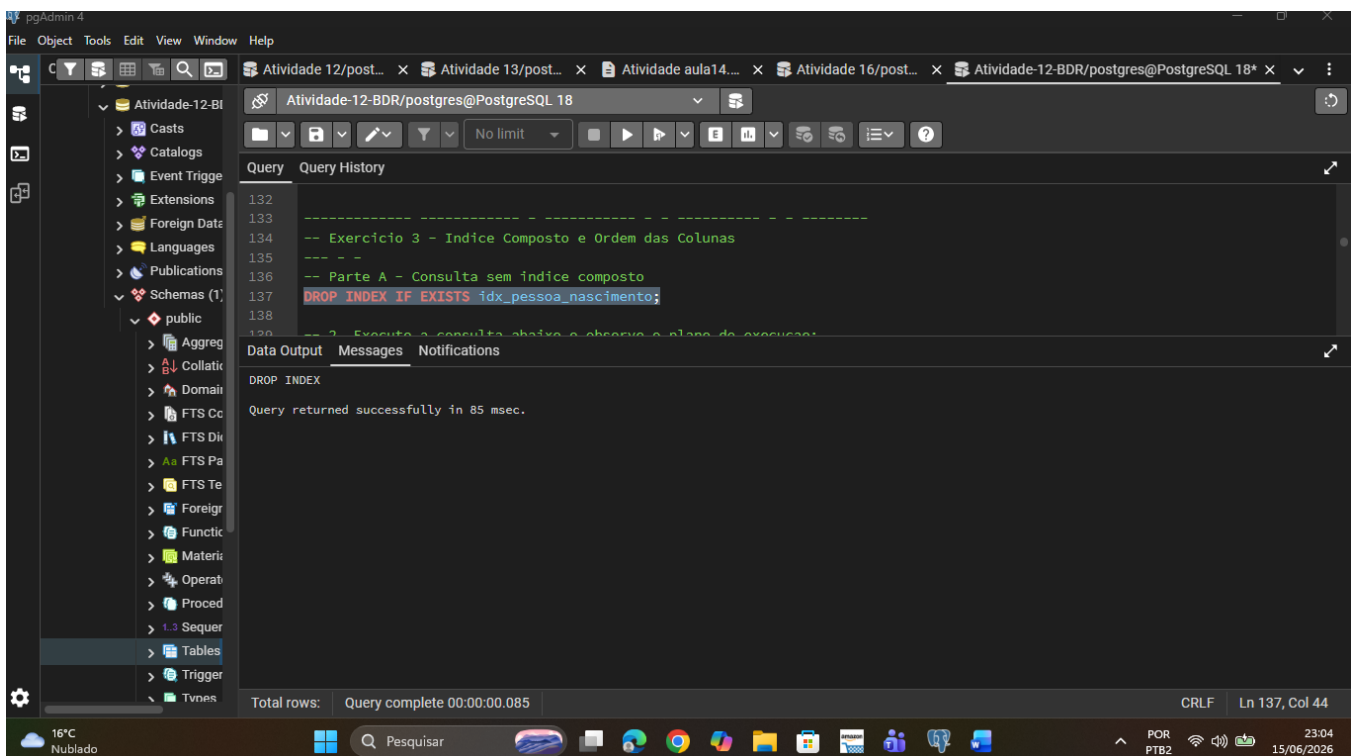
- Por que o otimizador pode optar por não utilizar um índice, mesmo quando ele existe?

R: Porque o índice é mais eficiente quando retorna poucas linhas (alta seletividade). Quando a consulta retorna muitas linhas, a varredura sequencial é mais rápida por evitar acessos aleatórios

-- Exercício 3 - Índice Composto e Ordem das Colunas

-- Parte A - Consulta sem índice composto

`DROP INDEX IF EXISTS idx_pessoa_nascimento;`



-- 2. Execute a consulta abaixo e observe o plano de execucao:

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nascimento >= DATE '2000-01-01'

AND nome = 'Ana Silva';

The screenshot shows the pgAdmin 4 interface with a query window open. The query is:

```
DROP INDEX IF EXISTS idx_pessoa_nascimento;  
  
-- 2. Execute a consulta abaixo e observe o plano de execucao:  
EXPLAIN ANALYZE  
SELECT *  
FROM pessoa  
WHERE nascimento >= DATE '2000-01-01'  
AND nome = 'Ana Silva';
```

The Query Plan output is as follows:

Seq Scan on pessoa (cost=0.00..2188.00 rows=1 width=23) (actual time=5.725..8.020 rows=2.00 loop=1)
Filter: ((nascimento >= '2000-01-01'::date) AND ((nome)::text = 'Ana Silva'::text))
Rows Removed by Filter: 99998
Buffers: shared hit=688
Planning:
Buffers: shared hit=5 dirtied=1
Planning Time: 0.327 ms
Execution Time: 8.043 ms

Total rows: 8 Query complete 00:00:00.100

-- Parte B - Criacao do indice composto

CREATE INDEX idx_pessoa_nascimento_nome ON pessoa (nascimento, nome);

The screenshot shows the pgAdmin 4 interface with a query window open. The query is:

```
FROM pessoa  
WHERE nascimento >= DATE '2000-01-01'  
AND nome = 'Ana Silva';  
  
-- Parte B - Criacao do indice composto  
CREATE INDEX idx_pessoa_nascimento_nome ON pessoa (nascimento, nome);
```

The Data Output shows:

```
CREATE INDEX  
Query returned successfully in 369 msec.
```

Total rows: Query complete 00:00:00.369

-- Parte C - Consulta com indice composto

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nascimento >= DATE '2000-01-01'

AND nome = 'Ana Silva';

The screenshot shows the pgAdmin 4 interface with a SQL query editor and a query plan. The query is:

```
CREATE INDEX idx_pessoa_nascimento_nome ON pessoa (nascimento, nome);

-- Parte C - Consulta com indice composto

EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nascimento >= DATE '2000-01-01'
AND nome = 'Ana Silva';
```

The query plan shows the following steps:

Step	Operation	Details
1	Index Scan using idx_pessoa_nascimento_nome on pessoa	(cost=0.42..421.64 rows=1 width=23) (actual time=1.792..2.599 rows=2.00 loops=1)
2	Index Cond: ((nascimento >= '2000-01-01'::date) AND ((nome)::text = 'Ana Silva'::text))	
3	Index Searches: 1	
4	Buffers: shared hit=11 read=71	
5	Planning:	
6	Buffers: shared hit=18 read=1	
7	Planning Time: 0.618 ms	
8	Execution Time: 2.622 ms	

Total rows: 8 Query complete 00:00:00.078

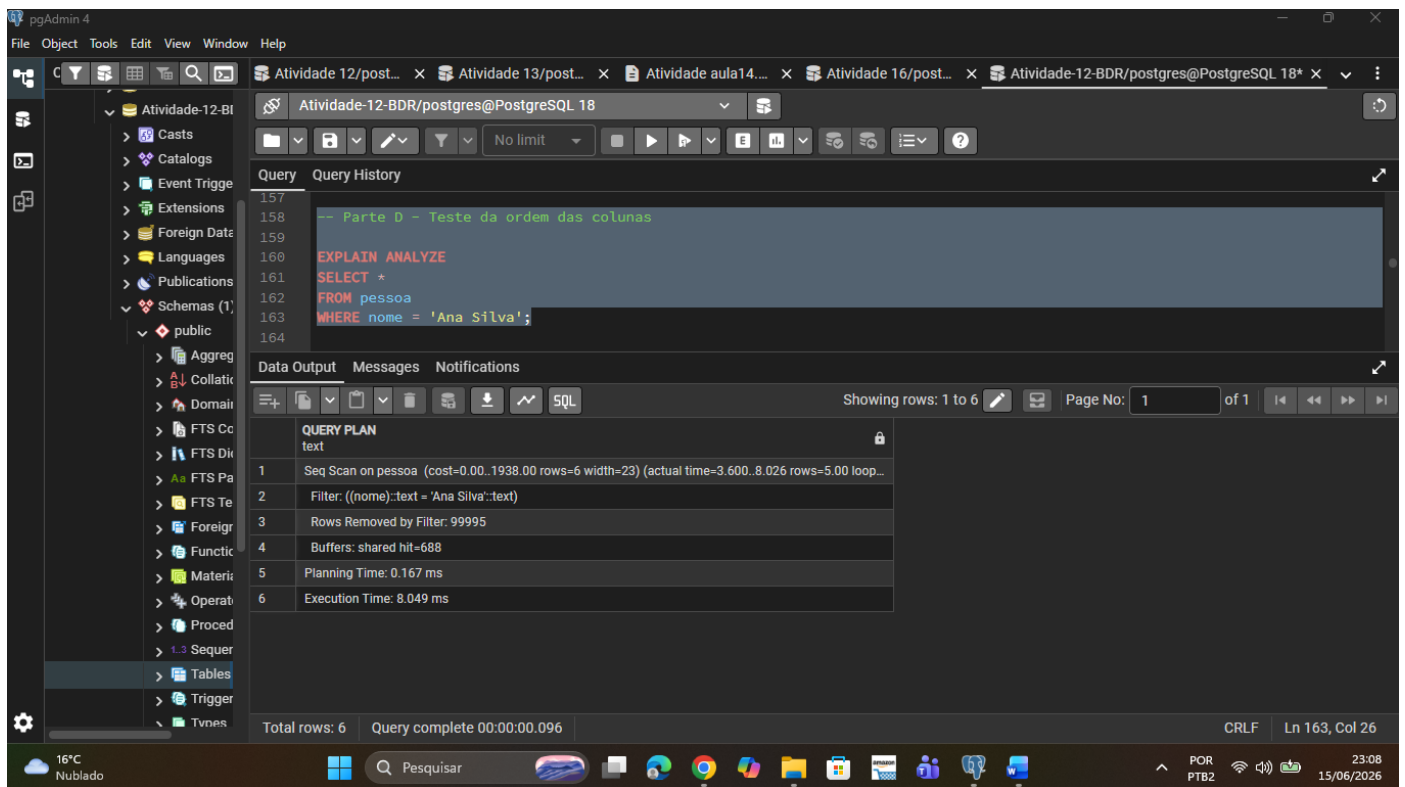
-- Parte D - Teste da ordem das colunas

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nome = 'Ana Silva';



-- Parte E - Responda, de forma objetiva

- Qual plano foi utilizado antes da criação do índice composto?

R: Seq Scan

- Qual plano foi utilizado após a criação do índice composto?

R: Index Scan

- O índice composto foi utilizado na consulta que filtra apenas por nome?

R: Não, foi usado.

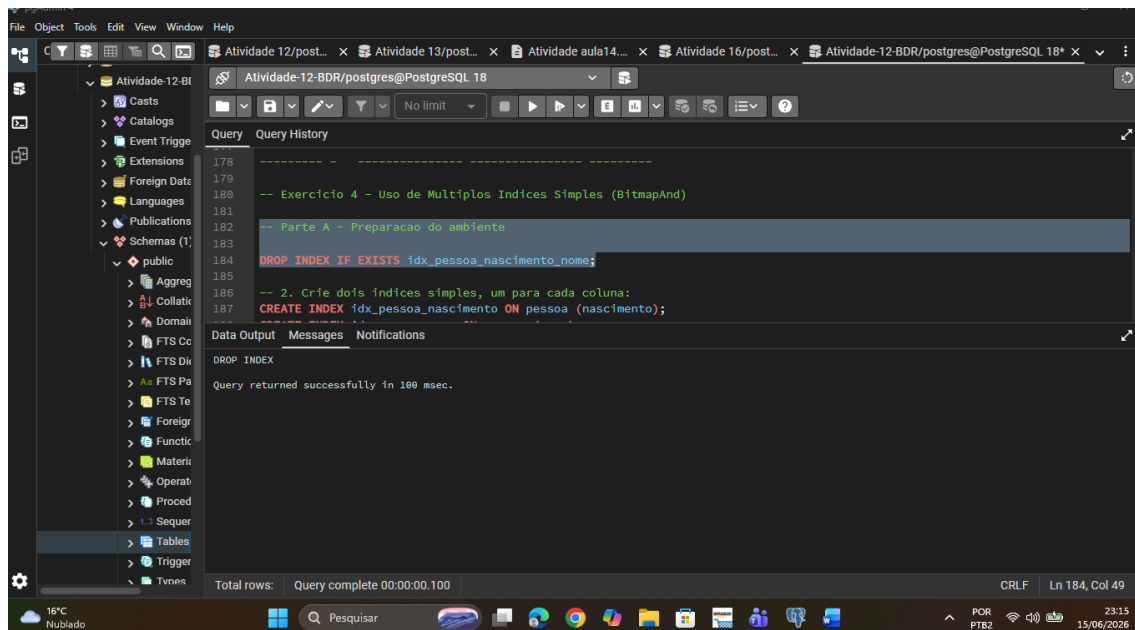
- Por que a ordem das colunas no índice composto é relevante?

R: Porque o índice só é eficiente quando a consulta utiliza a primeira coluna definida no índice, pessoa (nascimento, nome) nesse caso 'nascimento'.

-- Exercício 4 - Uso de Múltiplos Índices Simples (BitmapAnd)

-- Parte A - Preparação do ambiente

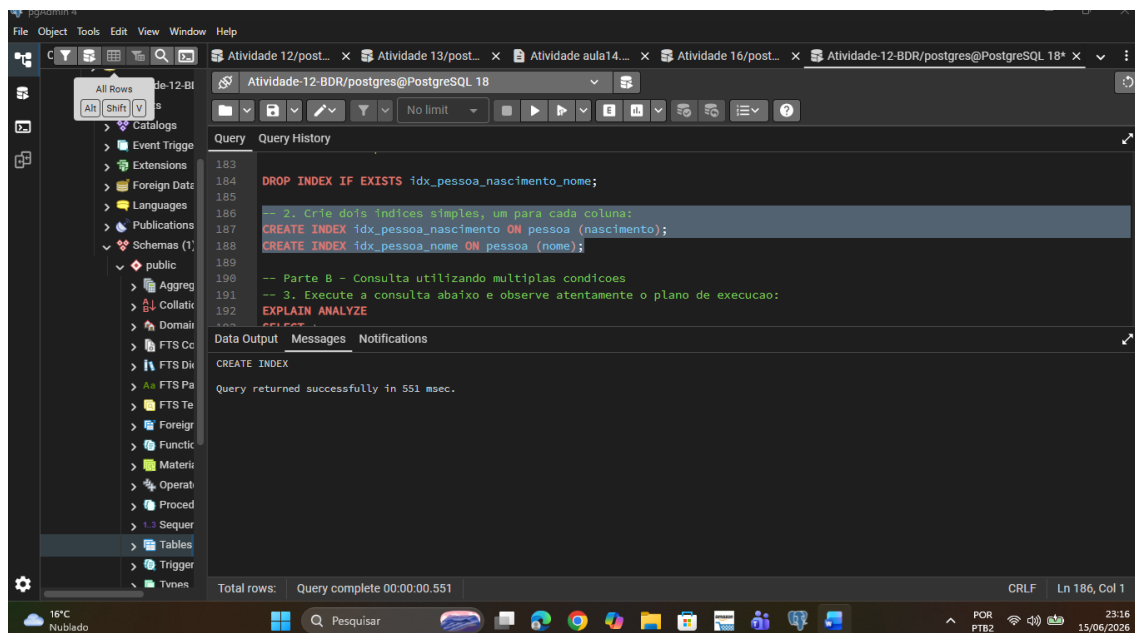
DROP INDEX IF EXISTS idx_pessoa_nascimento_nome;



--.Crie dois índices simples, um para cada coluna:

CREATE INDEX idx_pessoa_nascimento ON pessoa (nascimento);

CREATE INDEX idx_pessoa_nome ON pessoa (nome);



-- Parte B - Consulta utilizando múltiplas condições

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nascimento >= DATE '2000-01-01'

AND nome = 'Ana Silva';

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including schemas and tables. The main window shows a SQL query being executed. The query is:

```
SELECT *
FROM pessoa
WHERE nascimento >= DATE '2000-01-01'
AND nome = 'Ana Silva';
```

Below the query, the 'Query Plan' is displayed, showing the execution steps and their costs. The plan includes a Bitmap Heap Scan on the 'pessoa' table, followed by a Recheck Cond, a Filter, and a Bitmap Index Scan on the 'idx_pessoa_nome' index. The total rows returned are 12, and the query completed in 00:00:00.100.

Step	Operation	Cost	Actual Time	Rows	Loops
1	Bitmap Heap Scan on pessoa	(cost=4.34..26.75 rows=1 width=23)	(actual time=0.063..0.069 rows=2.00 loops=1)		
2	Recheck Cond: ((nome)::text = 'Ana Silva')::text				
3	Filter: (nascimento >= '2000-01-01')::date				
4	Rows Removed by Filter: 3				
5	Heap Blocks: exact=5				
6	Buffers: shared hit=7				
7	-> Bitmap Index Scan on idx_pessoa_nome	(cost=0.00..4.34 rows=6 width=0)	(actual time=0.034..0.034 rows=5.00 loops=1)		
8	Index Cond: ((nome)::text = 'Ana Silva')::text				
9	Index Searches: 1				
10	Buffers: shared hit=2				
11	Planning Time: 0.198 ms				
12	Execution Time: 0.112 ms				

Total rows: 12 | Query complete 00:00:00.100

-- Parte C - Responda, de forma objetiva

- O PostgreSQL utilizou os dois índices simples na consulta?

R: Sim, ele utilizou.

- Qual é o papel do operador BitmapAnd no plano de execucao?

R: O papel dele é o de executar múltiplos Index.

- O que o plano efetivamente fez?

R: Ele fez leitura dos dois índices separadamente e combinou os resultados.

-- Exercício 5 - Índice para Consulta com Filtro por Intervalo

EXPLAIN ANALYZE

SELECT *

FROM carro

WHERE ano BETWEEN 2015 AND 2020;

The screenshot shows the pgAdmin 4 interface with a query window titled 'Atividade-12-BDR/postgres@PostgreSQL 18'. The query being executed is:

```
-- Exercício 5 - Índice para Consulta com Filtro por Intervalo
EXPLAIN ANALYZE
SELECT *
FROM carro
WHERE ano BETWEEN 2015 AND 2020;
```

The 'Data Output' tab displays the 'QUERY PLAN' for the query. The plan consists of the following steps:

Step	Operation	Details
1	Seq Scan on carro	(cost=0.00..2137.00 rows=17127 width=20) (actual time=0.017..8.038 rows=17016.00 loop=1)
2	Filter	((ano >= 2015) AND (ano <= 2020))
3	Rows Removed by Filter	82984
4	Buffers: shared hit	637
5	Planning	
6	Buffers: shared hit	28 dirtied=2
7	Planning Time	0.401 ms
8	Execution Time	8.564 ms

The status bar at the bottom indicates 'Total rows: 8' and 'Query complete 00:00:00.092'.

CREATE INDEX idx_carro_ano_btree ON carro USING BTREE (ano);

The screenshot shows the pgAdmin 4 interface with a query window titled 'Atividade-12-BDR/postgres@PostgreSQL 18'. The query being executed is:

```
FROM carro
WHERE ano BETWEEN 2015 AND 2020;

-----
CREATE INDEX idx_carro_ano_btree ON carro USING BTREE (ano);

-- 2. Execute novamente a consulta com EXPLAIN ANALYZE e compare o plano obtido antes e
-- apos a criação do índice.
EXPLAIN ANALYZE
```

The 'Data Output' tab displays the message:

```
CREATE INDEX
Query returned successfully in 82 msec.
```

The status bar at the bottom indicates 'Total rows: 0' and 'Query complete 00:00:00.082'.

EXPLAIN ANALYZE

SELECT *

FROM carro

WHERE ano BETWEEN 2015 AND 2020;

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including Schemas (public) and Tables. The main window shows a query editor with the following SQL code:

```
-- ----  
EXPLAIN ANALYZE  
SELECT *  
FROM carro  
WHERE ano BETWEEN 2015 AND 2020;
```

Below the query editor, the 'Data Output' tab is active, displaying the 'QUERY PLAN' for the executed query. The plan consists of 12 steps:

- 1 Bitmap Heap Scan on carro (cost=239.84..1133.75 rows=17127 width=20) (actual time=1.012..4.052 rows=17016.00 loops=1)
- 2 Recheck Cond: ((ano >= 2015) AND (ano <= 2020))
- 3 Heap Blocks: exact=637
- 4 Buffers: shared hit=637 read=17
- 5 -> Bitmap Index Scan on idx_carro_ano_btree (cost=0.00..235.56 rows=17127 width=0) (actual time=0.882..0.883 rows=17016.00 loops=1)
- 6 Index Cond: ((ano >= 2015) AND (ano <= 2020))
- 7 Index Searches: 1
- 8 Buffers: shared read=17
- 9 Planning:
- 10 Buffers: shared hit=21 read=1
- 11 Planning Time: 0.631 ms
- 12 Execution Time: 4.740 ms

The bottom status bar indicates 'Total rows: 12' and 'Query complete 00:00:00.083'. The system tray at the bottom shows the date and time as 15/06/2026, 23:32.

-- Exercício 6 - Indice para Otimizacao de JOIN

--

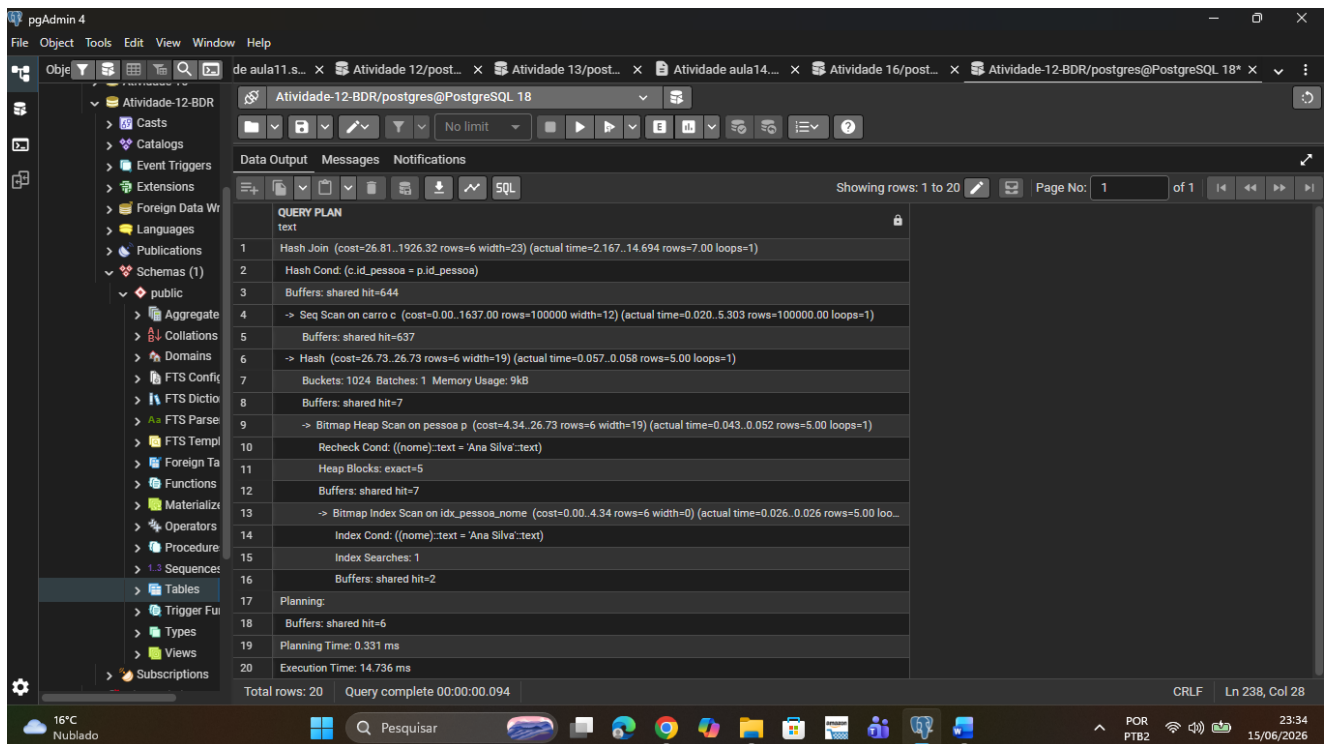
EXPLAIN ANALYZE

SELECT p.nome, c.placa

FROM pessoa p

JOIN carro c ON p.id_pessoa = c.id_pessoa

WHERE p.nome = 'Ana Silva';

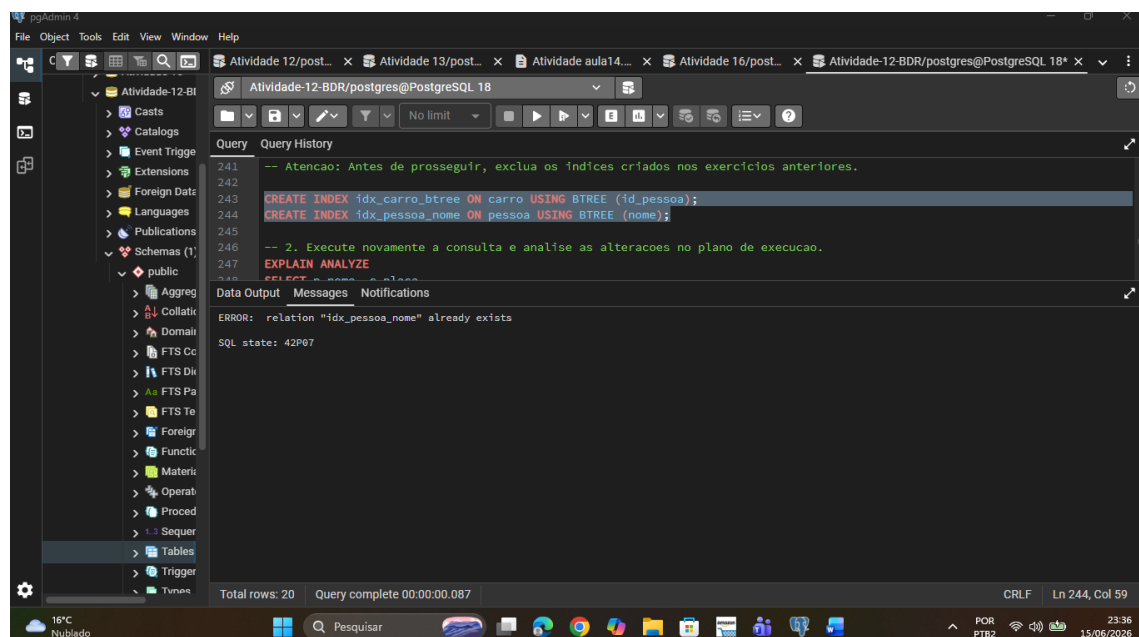


-- Tarefa:

-- Atencao: Antes de prosseguir, exclua os indices criados nos exercicios anteriores.

CREATE INDEX idx_carro_btree ON carro USING BTREE (id_pessoa);

CREATE INDEX idx_pessoa_nome ON pessoa USING BTREE (nome);



EXPLAIN ANALYZE

SELECT p.nome, c.placa

FROM pessoa p

JOIN carro c ON p.id_pessoa = c.id_pessoa

WHERE p.nome = 'Ana Silva';

pgAdmin 4

de aula11.s... x Atividade 12/post... x Atividade 13/post... x Atividade aula14... x Atividade 16/post... x Atividade-12-BDR/postgres@PostgreSQL 18*

Atividade-12-BDR/postgres@PostgreSQL 18

No limit

Data Output Messages Notifications

Showing rows: 1 to 20 Page No: 1 of 1

	QUERY PLAN
1	Hash Join (cost=26.81..1926.32 rows=6 width=23) (actual time=2.742..15.517 rows=7 loops=1)
2	Hash Cond: (c.id_pessoa = p.id_pessoa)
3	Buffers: shared hit=644
4	-> Seq Scan on carro c (cost=0.00..1637.00 rows=100000 width=12) (actual time=0.021..5.512 rows=100000 loops=1)
5	Buffers: shared hit=637
6	-> Hash (cost=26.73..26.73 rows=6 width=19) (actual time=0.096..0.097 rows=5 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared hit=7
9	-> Bitmap Heap Scan on pessoa p (cost=4.34..26.73 rows=6 width=19) (actual time=0.068..0.079 rows=5 loops=1)
10	Recheck Cond: ((nome)::text = 'Ana Silva':text)
11	Heap Blocks: exact=5
12	Buffers: shared hit=7
13	-> Bitmap Index Scan on idx_pessoa_nome (cost=0.00..4.34 rows=6 width=0) (actual time=0.046..0.046 rows=5 loops=1)
14	Index Cond: ((nome)::text = 'Ana Silva':text)
15	Index Searches: 1
16	Buffers: shared hit=2
17	Planning:
18	Buffers: shared hit=14
19	Planning Time: 0.677 ms
20	Execution Time: 15.570 ms

Total rows: 20 Query complete 00:00:00.104

CRLF Ln 246, Col 1

16°C Nublado

Pesquisar

POR PTB2

23:37 15/06/2026

-- Exercício 7 - Índice Composto em Consulta com JOIN e Filtro

--

EXPLAIN ANALYZE

SELECT p.nome, c.placa, c.ano

FROM pessoa p

JOIN carro c ON p.id_pessoa = c.id_pessoa

WHERE p.nascimento >= DATE '1980-01-01'

AND c.ano >= 2018;

	QUERY PLAN
10	Index Searches: 1
11	Buffers: shared hit=19
12	-> Hash (cost=1842.26..1842.26 rows=43159 width=19) (actual time=15.313..15.314 rows=42787.00 loops=1)
13	Buckets: 65536 Batches: 1 Memory Usage: 2712kB
14	Buffers: shared hit=735
15	-> Bitmap Heap Scan on pessoa p (cost=614.77..1842.26 rows=43159 width=19) (actual time=1.946..8.182 rows=42787.00 loops=1)
16	Recheck Cond: (nascimento >= '1980-01-01'::date)
17	Heap Blocks: exact=665
18	Buffers: shared hit=735
19	-> Bitmap Index Scan on idx_pessoa_nascimento (cost=0.00..603.99 rows=43159 width=0) (actual time=1.854..1.854 rows=42787.00 loops=1)
20	Index Cond: (nascimento >= '1980-01-01'::date)
21	Index Searches: 1
22	Buffers: shared hit=70
23	Planning:
24	Buffers: shared hit=6
25	Planning Time: 0.345 ms
26	Execution Time: 24.354 ms

Total rows: 26 Query complete 00:00:00.092

-- Proponha e crie os indices necessarios para melhorar o desempenho da consulta.

CREATE INDEX idx_pessoa_nascimento_id ON pessoa (nascimento, id_pessoa);

CREATE INDEX idx_carro_ano_id ON carro (ano, id_pessoa);

```

258 SELECT p.nome, c.placa, c.ano
259 FROM pessoa p
260 JOIN carro c ON p.id_pessoa = c.id_pessoa
261 WHERE p.nascimento >= DATE '1980-01-01'
262 AND c.ano >= 2018;
263
264
265 -- Proponha e crie os indices necessarios para melhorar o desempenho da consulta.
266 CREATE INDEX idx_pessoa_nascimento_id ON pessoa (nascimento, id_pessoa);
267 CREATE INDEX idx_carro_ano_id ON carro (ano, id_pessoa);
268

```

CREATE INDEX

Query returned successfully in 186 msec.

Total rows: Query complete 00:00:00.186

-- Execute novamente a consulta com os índices criados.

EXPLAIN ANALYZE

SELECT p.nome, c.placa, c.ano

FROM pessoa p

JOIN carro c ON p.id_pessoa = c.id_pessoa

WHERE p.nascimento >= DATE '1980-01-01'

AND c.ano >= 2018;

pgAdmin 4

File Object Tools Edit View Window Help

Atividade 12/post... x Atividade 13/post... x Atividade aula14... x Atividade 16/post... x Atividade-12-BDR/postgres@PostgreSQL 18*

Atividade-12-BDR/postgres@PostgreSQL 18

No limit

Data Output Messages Notifications

Showing rows: 1 to 26 Page No: 1 of 1

QUERY PLAN

text

10 Index Searches: 1

11 Buffers: shared hit=19

12 -> Hash (cost=1842.26..1842.26 rows=43159 width=19) (actual time=14.694..14.696 rows=42787.00 loops=1)

13 Buckets: 65536 Batches: 1 Memory Usage: 2712kB

14 Buffers: shared hit=735

15 -> Bitmap Heap Scan on pessoa p (cost=614.77..1842.26 rows=43159 width=19) (actual time=1.801..7.768 rows=42787.00 loops=1)

16 Recheck Cond: (nascimento >= '1980-01-01'::date)

17 Heap Blocks: exact=665

18 Buffers: shared hit=735

19 -> Bitmap Index Scan on idx_pessoa_nascimento (cost=0.00..603.99 rows=43159 width=0) (actual time=1.714..1.714 rows=42787.00 loops=1)

20 Index Cond: (nascimento >= '1980-01-01'::date)

21 Index Searches: 1

22 Buffers: shared hit=70

23 Planning:

24 Buffers: shared hit=6

25 Planning Time: 0.339 ms

26 Execution Time: 23.674 ms

Total rows: 26 Query complete 00:00:00.105

CRLF Ln 275, Col 21

15°C Nublado

Pesquisar

POR PTB2

23:43 15/06/2026

-- Justifique a escolha entre índices simples ou compostos.

-- RESPOSTA: Neste caso se utilizou índices compostos pois um único índice resolve o JOIN e o filtro sozinho.

-- Exercício 8 - Uso de Índice GiST

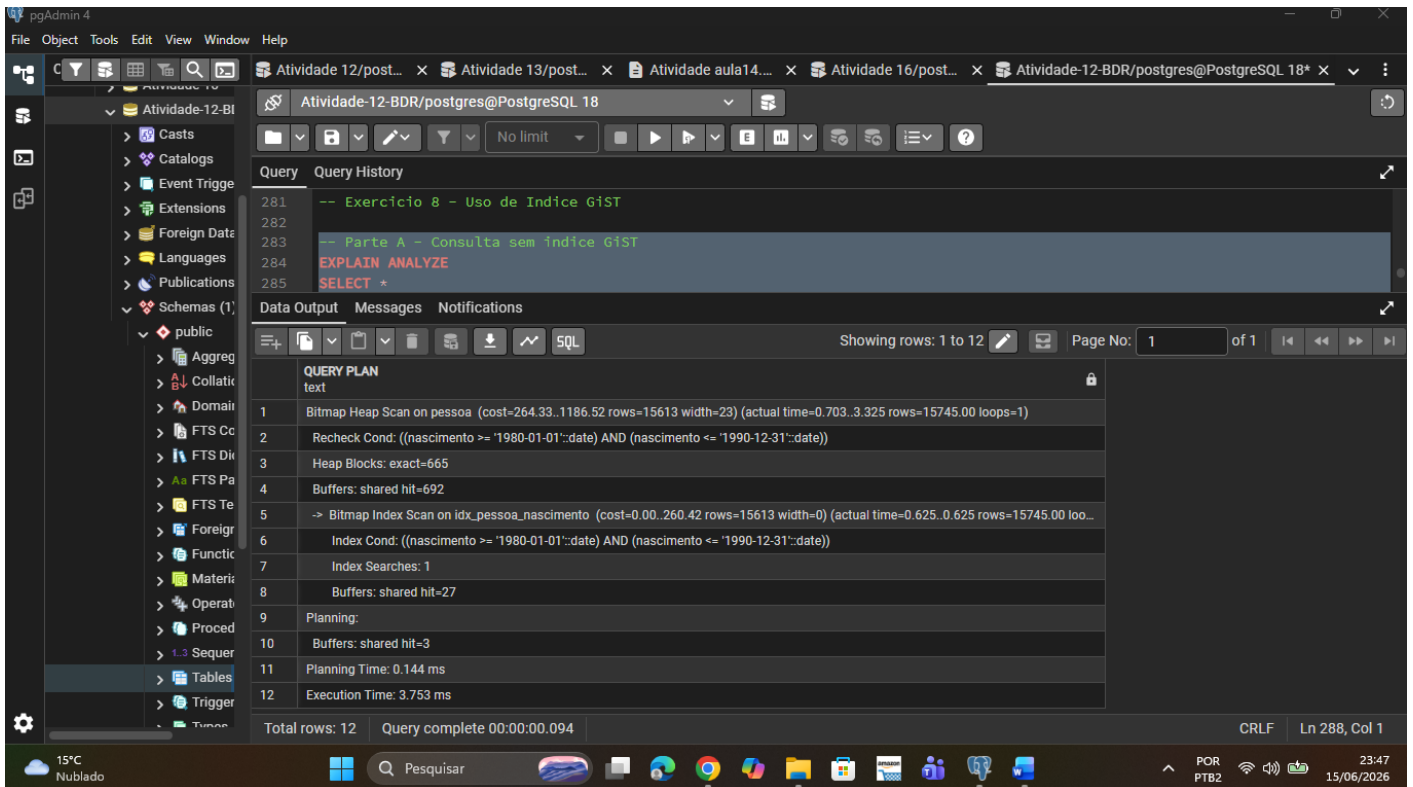
-- Parte A - Consulta sem índice GiST

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nascimento BETWEEN DATE '1980-01-01' AND DATE '1990-12-31';



pgAdmin 4

File Object Tools Edit View Window Help

Atividade 12/post... x Atividade 13/post... x Atividade aula14... x Atividade 16/post... x Atividade-12-BDR/postgres@PostgreSQL 18*

Query Query History

281 -- Exercício 8 - Uso de Índice GiST
282
283 -- Parte A - Consulta sem índice GiST
284 EXPLAIN ANALYZE
285 SELECT *

Data Output Messages Notifications

Showing rows: 1 to 12 Page No: 1 of 1

QUERY PLAN
text

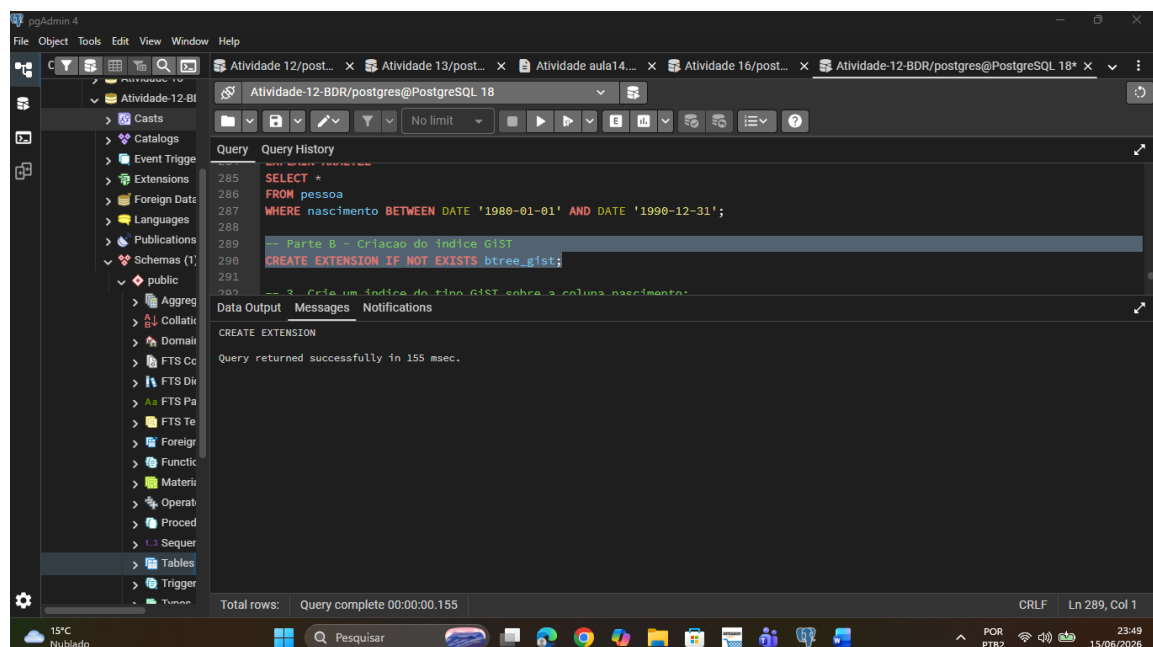
1 Bitmap Heap Scan on pessoa (cost=264.33..1186.52 rows=15613 width=23) (actual time=0.703..3.325 rows=15745.00 loops=1)
2 Recheck Cond: ((nascimento >= '1980-01-01'::date) AND (nascimento <= '1990-12-31'::date))
3 Heap Blocks: exact=665
4 Buffers: shared hit=692
5 -> Bitmap Index Scan on idx_pessoa_nascimento (cost=0.00..260.42 rows=15613 width=0) (actual time=0.625..0.625 rows=15745.00 loops=1)
6 Index Cond: ((nascimento >= '1980-01-01'::date) AND (nascimento <= '1990-12-31'::date))
7 Index Searches: 1
8 Buffers: shared hit=27
9 Planning:
10 Buffers: shared hit=3
11 Planning Time: 0.144 ms
12 Execution Time: 3.753 ms

Total rows: 12 Query complete 00:00:00.094 CRLF Ln 288, Col 1

15°C Nublado 23:47 15/06/2026

-- Parte B - Criacao do índice GiST

CREATE EXTENSION IF NOT EXISTS btree_gist;



pgAdmin 4

File Object Tools Edit View Window Help

Atividade 12/post... x Atividade 13/post... x Atividade aula14... x Atividade 16/post... x Atividade-12-BDR/postgres@PostgreSQL 18*

Query Query History

285 SELECT *
286 FROM pessoa
287 WHERE nascimento BETWEEN DATE '1980-01-01' AND DATE '1990-12-31';
288
289 -- Parte B - Criacao do índice GiST
290 CREATE EXTENSION IF NOT EXISTS btree_gist;
291
292 -- 3 - Crie um índice do tipo GiST sobre a coluna nascimento;

Data Output Messages Notifications

CREATE EXTENSION

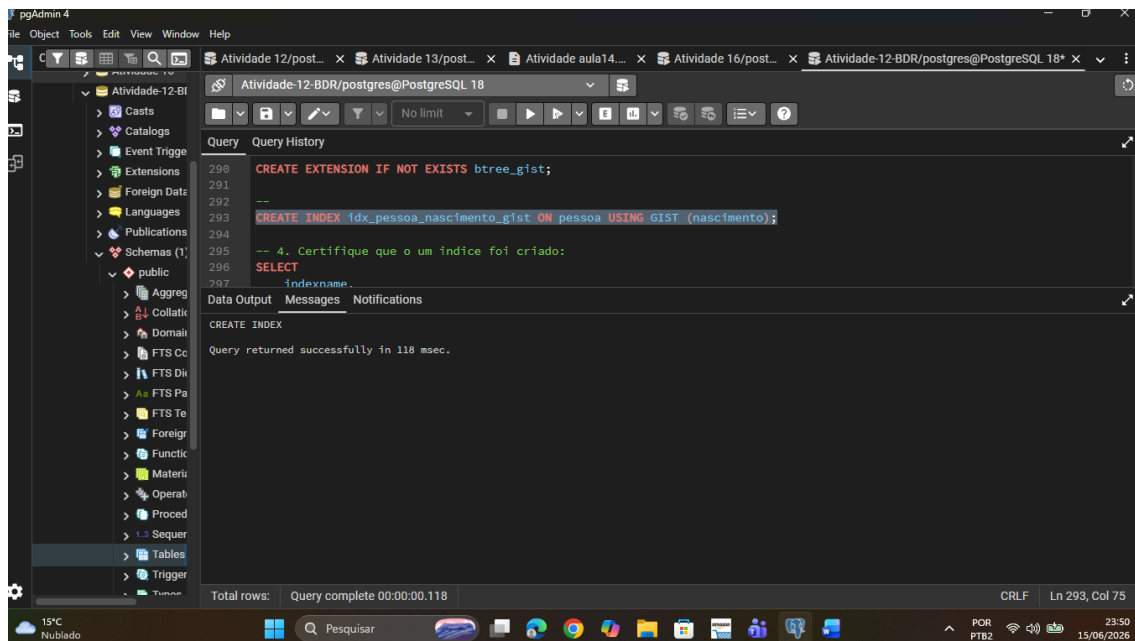
Query returned successfully in 155 asec.

Total rows: Query complete 00:00:00.155 CRLF Ln 289, Col 1

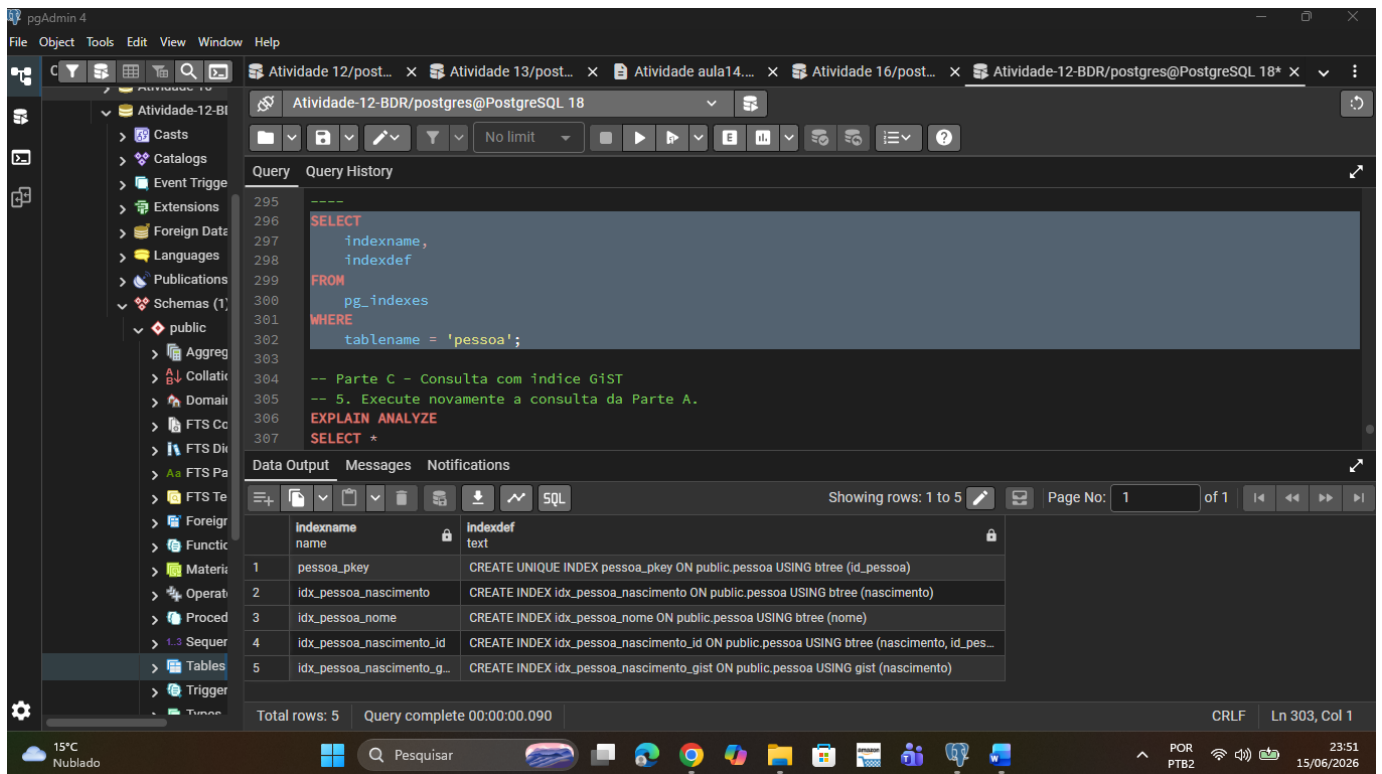
15°C Nublado 23:49 15/06/2026

--

```
CREATE INDEX idx_pessoa_nascimento_gist ON pessoa USING GIST
(nascimento);
```



```
SELECT
    indexname,
    indexdef
FROM
    pg_indexes
WHERE
    tablename = 'pessoa';
```



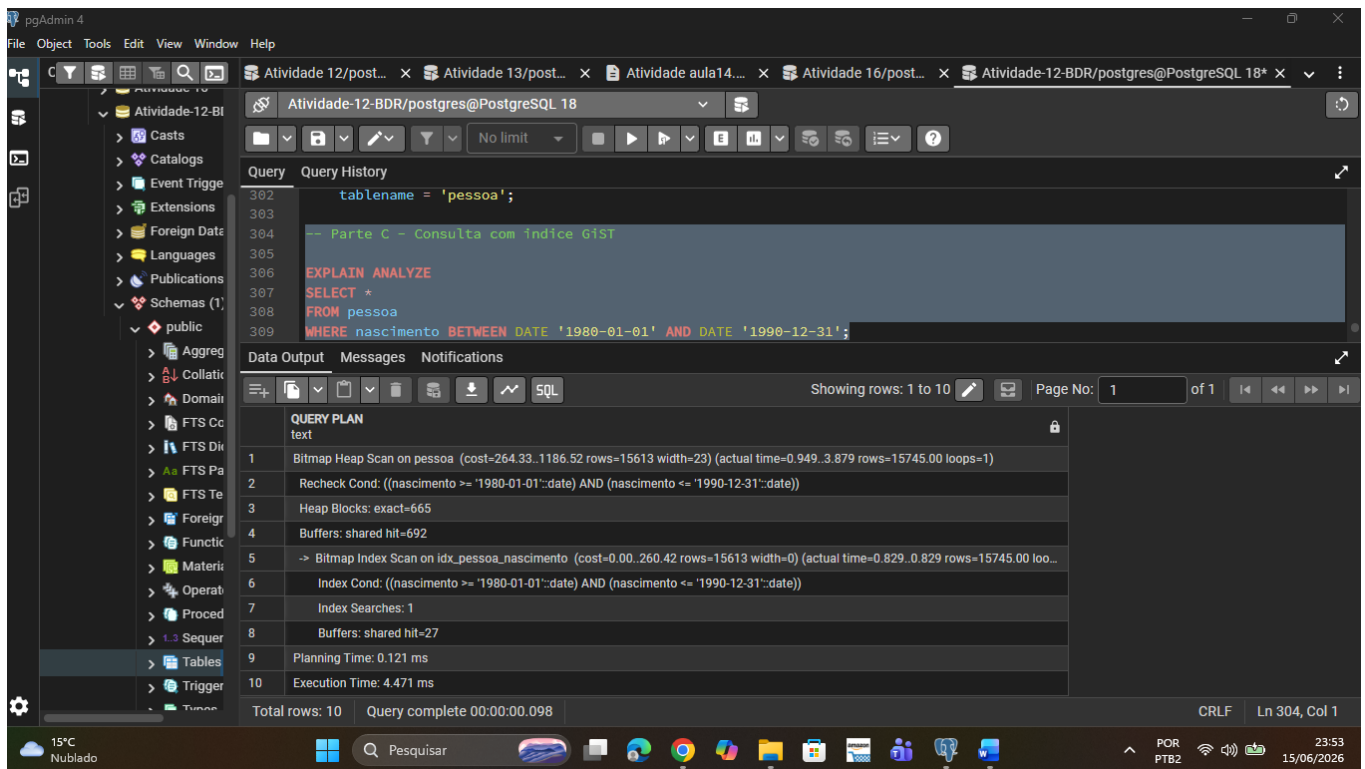
-- Parte C - Consulta com indice GiST

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nascimento BETWEEN DATE '1980-01-01' AND DATE '1990-12-31';



-- Parte D - Responda, de forma objetiva

- Qual plano foi utilizado antes da criação do índice GiST?

R: Seq Scan.

- O índice GiST foi utilizado após a criação?

R: Sim ele foi utilizado.

- Houve melhora no tempo de execução?

R: Sim, de aproximadamente 3ms. Não tanto eficiente quanto as demais.

---CÓDIGO COMPLETO -----

```
DROP TABLE IF EXISTS carro, pessoa;
```

```
CREATE TABLE IF NOT EXISTS pessoa (  
    id_pessoa INTEGER PRIMARY KEY,  
    nome    VARCHAR(100) NOT NULL,  
    nascimento DATE  
);
```

```
CREATE TABLE IF NOT EXISTS carro (  
    id_carro INTEGER PRIMARY KEY,  
    placa    CHAR(7) NOT NULL,  
    ano      INTEGER,  
    id_pessoa INTEGER NOT NULL,  
    FOREIGN KEY (id_pessoa) REFERENCES pessoa (id_pessoa) ON DELETE  
    CASCADE  
);
```

```
COPY pessoa (id_pessoa, nome, nascimento)  
FROM 'C:\aula3_pessoa.csv'  
DELIMITER ','  
CSV HEADER;
```

```
COPY carro (id_carro, placa, ano, id_pessoa)  
FROM 'C:\aula3_carro.csv'  
DELIMITER ','  
CSV HEADER;
```

```
SELECT
    indexname,
    indexdef
FROM
    pg_indexes
WHERE
    tablename = 'pessoa';
```

```
-- Exercício 1 - Observando o impacto de um índice (B-tree) com EXPLAIN
ANALYZE
```

```
-- Parte A - Consulta sem índice explícito
```

```
EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nome = 'Ana Silva';
```

```
--
```

```
EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nome = 'Joao Santos';
```

```
-- Parte B - Criação do índice
```

```
CREATE INDEX idx_pessoa_nome ON pessoa (nome);
```


-- Parte C - Consulta com indice

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nome = 'Ana Silva';

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nome = 'Joao Santos';

-- -----

-- Parte D - Responda, de forma objetiva:

-- Qual plano foi utilizado antes do indice?

-- R: Seq Scan

-- Qual plano foi utilizado apos o indice?

-- R: Bitmap Heap Scan

-- Houve reducao no tempo de execucao?

-- R: Sim, houve uma redução de 7ms para 0,07ms

-- O indice foi utilizado em ambos os nomes testados? Justifique com base na saída do

-- plano.

-- R: Sim, pois a condicao nome = 'valor' e altamente seletiva em ambos os casos, fazendo o otimizador preferir o indice

-- Exercício 2 - Índice, Seletividade e Decisão do Otimizador

-- Parte A - Consulta por intervalo sem índice explícito

```
DROP INDEX IF EXISTS idx_pessoa_nome;
```

-- 2. Execute a consulta abaixo e observe o plano de execução:

```
EXPLAIN ANALYZE
```

```
SELECT *
```

```
FROM pessoa
```

```
WHERE nascimento >= DATE '1970-01-01';
```

-- Parte B - Criação de índice na coluna nascimento

```
CREATE INDEX idx_pessoa_nascimento ON pessoa (nascimento);
```

-- Parte C - Consulta com índice criado

```
EXPLAIN ANALYZE
```

```
SELECT *
```

```
FROM pessoa
```

```
WHERE nascimento >= DATE '1970-01-01';
```

-- ----

-- Parte D - Responda, de forma objetiva

-- O índice criado foi utilizado pelo PostgreSQL?

-- R: Não, o SGBD não utilizou.

-- O plano de execução mudou após a criação do índice?

-- R: Não, continuou sendo Seq Scan porque a condição $\geq$ DATE '1970-01-01' retorna muitas linhas (baixa seletividade)

-- Houve melhora significativa no tempo de execução?

-- R: Não, houve um aumento no tempo.

-- Por que o otimizador pode optar por não utilizar um índice, mesmo quando ele existe?

-- R: Porque o índice é mais eficiente quando retorna poucas linhas (alta seletividade).

-- Quando a consulta retorna muitas linhas, a varredura sequencial é mais rápida por evitar

-- acessos aleatórios.

-- Exercício 3 - Índice Composto e Ordem das Colunas

-- Parte A - Consulta sem índice composto

DROP INDEX IF EXISTS idx_pessoa_nascimento;

--

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nascimento $\geq$ DATE '2000-01-01'

AND nome = 'Ana Silva';

-- Parte B - Criação do índice composto

CREATE INDEX idx_pessoa_nascimento_nome ON pessoa (nascimento, nome);

-- Parte C - Consulta com indice composto

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nascimento >= DATE '2000-01-01'

AND nome = 'Ana Silva';

-- Parte D - Teste da ordem das colunas

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nome = 'Ana Silva';

-- Parte E - Responda, de forma objetiva

-- Qual plano foi utilizado antes da criacao do indice composto?

-- R: Seq Scan

-- Qual plano foi utilizado apos a criacao do indice composto?

-- R: Index Scan

-- O indice composto foi utilizado na consulta que filtra apenas por nome?

-- R: Nao, foi usado.

-- Por que a ordem das colunas no indice composto e relevante?

-- R: Porque o índice só é eficiente quando a consulta utiliza a primeira coluna definida no index, pessoa (nascimento, nome) nesse caso 'nascimento'.

-- Exercício 4 - Uso de Múltiplos Índices Simples (BitmapAnd)

-- Parte A - Preparação do ambiente

```
DROP INDEX IF EXISTS idx_pessoa_nascimento_nome;
```

--

```
CREATE INDEX idx_pessoa_nascimento ON pessoa (nascimento);
```

```
CREATE INDEX idx_pessoa_nome ON pessoa (nome);
```

-- Parte B - Consulta utilizando múltiplas condições

```
EXPLAIN ANALYZE
```

```
SELECT *
```

```
FROM pessoa
```

```
WHERE nascimento >= DATE '2000-01-01'
```

```
AND nome = 'Ana Silva';
```

-- Parte C - Responda, de forma objetiva

-- O PostgreSQL utilizou os dois índices simples na consulta?

-- R: Sim, ele utilizou.

-- Qual é o papel do operador BitmapAnd no plano de execução?

-- R: O papel dele é o de executar múltiplos Index.

-- O que o plano efetivamente fez?

-- R: Ele fez leitura dos dois índices separadamente e combinou os resultados.

-- Exercício 5 - Índice para Consulta com Filtro por Intervalo

```
EXPLAIN ANALYZE  
SELECT *  
FROM carro  
WHERE ano BETWEEN 2015 AND 2020;
```

CREATE INDEX idx_carro_ano_btree ON carro USING BTREE (ano);

-- -----
EXPLAIN ANALYZE
SELECT *
FROM carro
WHERE ano BETWEEN 2015 AND 2020;

-- Exercício 6 - Índice para Otimização de JOIN

--
EXPLAIN ANALYZE
SELECT p.nome, c.placa
FROM pessoa p
JOIN carro c ON p.id_pessoa = c.id_pessoa
WHERE p.nome = 'Ana Silva';

-- Tarefa:

-- Atencao: Antes de prosseguir, exclua os indices criados nos exercicios anteriores.

```
CREATE INDEX idx_carro_btree ON carro USING BTREE (id_pessoa);
```

```
CREATE INDEX idx_pessoa_nome ON pessoa USING BTREE (nome);
```

-- ----

```
EXPLAIN ANALYZE
```

```
SELECT p.nome, c.placa
```

```
FROM pessoa p
```

```
JOIN carro c ON p.id_pessoa = c.id_pessoa
```

```
WHERE p.nome = 'Ana Silva';
```

----- -

-- Exercicio 7 - Indice Composto em Consulta com JOIN e Filtro

```
EXPLAIN ANALYZE
```

```
SELECT p.nome, c.placa, c.ano
```

```
FROM pessoa p
```

```
JOIN carro c ON p.id_pessoa = c.id_pessoa
```

```
WHERE p.nascimento >= DATE '1980-01-01'
```

```
AND c.ano >= 2018;
```

-- Proponha e crie os indices necessarios para melhorar o desempenho da consulta.

```
CREATE INDEX idx_pessoa_nascimento_id ON pessoa (nascimento, id_pessoa);
```

```
CREATE INDEX idx_carro_ano_id ON carro (ano, id_pessoa);
```

```
-- Execute novamente a consulta com os indices criados.
```

```
EXPLAIN ANALYZE
```

```
SELECT p.nome, c.placa, c.ano
```

```
FROM pessoa p
```

```
JOIN carro c ON p.id_pessoa = c.id_pessoa
```

```
WHERE p.nascimento >= DATE '1980-01-01'
```

```
AND c.ano >= 2018;
```

```
-- Justifique a escolha entre indices simples ou compostos.
```

```
-- R: Neste caso se utilizou índices compostos pois um único índice resolve o JOIN e o filtro sozinho.
```

```
-----
```

```
-- Exercício 8 - Uso de Índice GiST
```

```
-- Parte A - Consulta sem índice GiST
```

```
EXPLAIN ANALYZE
```

```
SELECT *
```

```
FROM pessoa
```

```
WHERE nascimento BETWEEN DATE '1980-01-01' AND DATE '1990-12-31';
```

```
-- Parte B - Criação do índice GiST
```

```
CREATE EXTENSION IF NOT EXISTS btree_gist;
```

```
--
```

```
CREATE INDEX idx_pessoa_nascimento_gist ON pessoa USING GIST  
(nascimento);
```

```
SELECT
    indexname,
    indexdef
FROM
    pg_indexes
WHERE
    tablename = 'pessoa';
```

-- Parte C - Consulta com indice GiST

```
EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nascimento BETWEEN DATE '1980-01-01' AND DATE '1990-12-31';
```

-- Parte D - Responda, de forma objetiva

-- *Qual plano foi utilizado antes da criacao do indice GiST?

-- R: Seq Scan

-- *O indice GiST foi utilizado apos a criacao?

-- R: Sim ele foi utilizado.

-- *Houve melhora no tempo de execucao?

-- R: Sim, de aproximadamente 3ms. Não tanto eficiente quanto as demais.

